

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Dilliraj N(192324249) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Dilliraj N

Signature of the student:

Annexure A

Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

Python code

```
import hashlib

messages = ["Hello", "Python", "Cryptography", "Security"]

for msg in messages:
    md5 = hashlib.md5(msg.encode()).hexdigest()
    sha256 = hashlib.sha256(msg.encode()).hexdigest()

    print("\nMessage:", msg)
    print("MD5      :", md5)
    print("SHA256   :", sha256)

print("\nComparison:")
print("MD5      : 128-bit hash, faster but weak against collisions")
print("SHA256   : 256-bit hash, stronger collision resistance")
```

Output

Message:

Hello

MD5 : 8b1a9953c4611296a827abf8c47804d7

SHA256 : 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

Message:

Python

MD5 : a7f5f35426b927411fc9231b56382173

SHA256 : 18885f27b5af9012df19e496460f9294d5ab76128824c6f993787004f6d9a7db

Message:

Cryptography

MD5 : 64ef07ce3e4b420c334227eecb3b3f4c

SHA256 : b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857

Message:

Security

MD5 : 2fae32629d4ef4fc6341f1751b405e45

SHA256 : 8f6fb4eb7f42c0e245e29e63f5b82cc3ba19852681d1ed9aed291f59cf75ec0e

Comparison:

MD5 : 128-bit hash, faster but weak against collisions

SHA256 : 256-bit hash, stronger collision resistance

Run completed in 4576.5ms

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

Python code

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate RSA keys
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

# Message
message = b"Hello, this is a secure message"

# Sign message
signature = private_key.sign(
    message,
    padding.PKCS1v15(),
    hashes.SHA256()
)

print("Message:", message)
print("Signature created successfully")

# Verify signature
try:
    public_key.verify(
        signature,
        message,
        padding.PKCS1v15(),
        hashes.SHA256()
    )
    print("Signature Verification: SUCCESS")
except:
    print("Signature Verification: FAILED")

# Change message
modified_message = b"Hello, modified message"

try:
    public_key.verify(
        signature,
        modified_message,
        padding.PKCS1v15(),
```

Output

```
Message: b'Hello, this is a secure message'  
Signature created successfully  
Signature Verification: SUCCESS  
Modified message rejected  
Run completed in 4740.200000000186ms
```

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

Python code

```
import hmac
import hashlib

# Message and secret key
message = b"Hello, this is a secure message"
key = b"secret123"

# Create HMAC
h = hmac.new(key, message, hashlib.sha256).hexdigest()

print("Message:", message)
print("HMAC-SHA256:", h)

# Verify HMAC
new_h = hmac.new(key, message, hashlib.sha256).hexdigest()

if hmac.compare_digest(h, new_h):
    print("HMAC Verification: SUCCESS")
else:
    print("HMAC Verification: FAILED")

# Modified message
modified = b"Hello, modified message"

new_h = hmac.new(key, modified, hashlib.sha256).hexdigest()

if hmac.compare_digest(h, new_h):
    print("Modified message is valid")
else:
    print("Modified message rejected")
```

Output

```
Message: b'Hello, this is a secure message'
HMAC-SHA256: 4ea026bbb00882ccb61509cbcfaba5dcd6329551b80ab0964d4925b0c4c34c28
HMAC Verification: SUCCESS
Modified message rejected
Run completed in 6212.200000000186ms
```

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

Python code

```
import time
import secrets
import hmac
import hashlib

# Secret key of server
key = b"server_secret"

# Client
username = "Alice"

# Generate ticket
ticket_id = secrets.token_hex(4)
timestamp = int(time.time())

ticket = username + "|" + ticket_id + "|" + str(timestamp)

# Sign ticket
signature = hmac.new(
    key,
    ticket.encode(),
    hashlib.sha256
).hexdigest()

print("Client:", username)
print("Ticket:", ticket)
print("Ticket generated successfully")

# Server validates ticket
received_signature = hmac.new(
    key,
    ticket.encode(),
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(signature, received_signature):
    print("Ticket Verification: SUCCESS")
else:
    print("Ticket Verification: FAILED")

# Check ticket time
current_time = int(time.time())

if current_time - timestamp <= 30:
```

Output

```
Client: Alice
Ticket: Alice|dea61cfa|1786715990
Ticket generated successfully
Ticket Verification: SUCCESS
Ticket is valid

Replay Attack:
Same ticket used again
Replay can be detected using the ticket ID and used-ticket list.
Run completed in 4828.4000000003725ms
```

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

Python code

```
import hashlib
import secrets

# Small values for demonstration only
p = 23
q = 11
g = 2

# Generate private key
private_key = secrets.randbelow(q - 1) + 1

# Generate public key
public_key = pow(g, private_key, p)

print("Private Key:", private_key)
print("Public Key:", public_key)

# Message
message = "Hello World"

# Generate random number
k = secrets.randbelow(q - 1) + 1

# Calculate r
r = pow(g, k, p)

# Calculate hash
e = int(
    hashlib.sha256(
        (message + str(r)).encode()
    ).hexdigest(),
    16
) % q

# Generate signature
s = (k + private_key * e) % q

print("\nMessage:", message)
print("Signature:", (e, s))

# Verification
r2 = (
    pow(g, s, p) *
    pow(pow(public_key, e, p), p - 2, p)
) % p
```

output

```
Private Key: 7
Public Key: 13

Message:
  Hello World
Signature: (3, 0)
Signature Verification: SUCCESS
Modified message rejected
Run completed in 4615.100000000559ms
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation